

Université Hassan 1^{er}
École Nationale des Sciences Appliquées de Berrechid
Département : Génie Informatique et Mathématiques
Filière : Systèmes d'Information et Big Data

Rapport de Projet 9

Contrôleur de chaîne logistique avec analyse géospatiale

Réalisé par :

ABARKHA Fatima Ezzahra
MOUAKI Maryam

Encadré par :

Pr. HRIMECH

Résumé

Ce projet présente la conception et le déploiement d'un système intelligent de supervision et d'optimisation logistique fondé sur l'exploitation de données géospatiales. À partir d'un jeu de données GPS réelles (trajets de taxis à Porto, janvier 2014), la plateforme simule un flux temps réel via **Kafka** (Apache Kafka), archive les positions dans une base spatiale **PostGIS** (PostgreSQL/PostGIS), et propose un tableau de bord interactif développé avec **Streamlit**.

L'architecture couvre l'ensemble du cycle de valeur décisionnel : prétraitement des trajectoires, ingestion en streaming, clustering spatial (**DBSCAN** (Density-Based Spatial Clustering of Applications with Noise)), enrichissement météorologique (**Open-Meteo** (API Open-Meteo)), calcul d'un score de risque logistique, prévision des retards, optimisation heuristique des tournées, publication cartographique avec **GeoServer**, visualisation *heatmap*, isochrones et export des couches géographiques vers **Kepler.gl**.

Les résultats démontrent une réduction moyenne de 3.98 % sur les distances parcourues, un gain de 1172.9 min sur les temps de trajet, et une capacité opérationnelle à identifier les zones critiques, anticiper les perturbations météorologiques et prioriser les interventions. Le système constitue un prototype opérationnel facilement extensible vers des environnements de production cloud.

Mots-clés : logistique, analyse géospatiale, Kafka, PostGIS, Streamlit, DBSCAN, Open-Meteo, GeoServer, WMS, WFS, heatmap, isochrones, Kepler.gl, optimisation, score de risque, aide à la décision.

Table des matières

Résumé	1
1 Introduction générale	7
1.1 Contexte du projet	7
1.2 Problématique	7
1.3 Objectifs du projet	7
1.4 Organisation du rapport	8
2 Données utilisées	9
2.1 Description du jeu de données	9
2.2 Colonnes principales	9
2.3 Préparation et validation	9
2.4 Indicateurs de qualité	10
3 Technologies utilisées	11
3.1 Architecture logicielle	11
3.2 Composants principaux	11
3.3 Justifications techniques	12
4 Architecture du système	13
4.1 Vue macroscopique	13
4.2 Pipeline analytique	14
4.3 Séquence du flux temps réel	14
5 Base de données PostGIS	16

5.1	Rôle et modélisation	16
5.2	Tables principales	16
5.3	Exemple de DDL	16
6	Simulation temps réel avec Kafka	18
6.1	Principe de fonctionnement	18
6.2	Paramétrage et robustesse	18
6.3	Latence et performance	18
7	Tableau de bord Streamlit	19
7.1	Conception ergonomique	19
7.2	Pages fonctionnelles et détails techniques	20
7.2.1	Vue globale	20
7.2.2	Page temps réel	21
7.2.3	Page Alertes temps réel	22
7.2.4	Page Heatmap	23
7.2.5	Page tournées	24
7.2.6	Page Clusters DBSCAN	25
7.2.7	Page Score de risque	26
7.2.8	Page Prévisions	27
7.2.9	Page Météo Open-Meteo	28
7.2.10	Page GeoServer	30
7.2.11	Page Isochrones	31
7.2.12	Page Export Kepler.gl	32
8	Guide de déploiement et d'exécution	34
8.1	Prérequis	34
8.2	Orchestration des services	34
8.3	Pipeline de traitement	34
8.4	Lancement automatique de la démonstration	35
8.4.1	Script temps réel	35

8.4.2	Script de recalcul analytique	35
9	Étude de cas et analyse métier	37
9.1	Scénario opérationnel	37
9.2	Détection de la zone critique	37
9.3	Impact de la météo	37
9.4	Priorisation des véhicules	38
9.5	Action proposée	38
9.6	Synthèse métier	38
10	Résultats et indicateurs de performance	39
11	Limites et perspectives	40
11.1	Limites actuelles	40
11.2	Feuille de route	40
12	Conclusion	41
A	Structure du projet	42

Table des figures

4.1	Architecture générale du système	13
4.2	Pipeline analytique et dépendances entre modules	14
4.3	Diagramme de séquence du flux Kafka → PostGIS	15
7.1	Vue globale du tableau de bord (synthèse exécutive)	20
7.2	Page Temps réel et véhicules prioritaires	21
7.3	Page Alertes temps réel	22
7.4	Page Heatmap	23
7.5	Page Tournées et gains d'optimisation	24
7.6	Page Clusters DBSCAN	25
7.7	Page Score de risque	26
7.8	Page Prévisions des retards	27
7.9	Page Météo Open-Meteo	28
7.10	Page GeoServer	30
7.11	Page Isochrones	31
7.12	Page Export Kepler.gl	32
7.13	Superposition des couches dans Kepler.g	33

Liste des tableaux

2.1	Structure du jeu de données source	9
2.2	Métriques issues du prétraitement	10
3.1	Stack technologique et rôle opérationnel	11
5.1	Schéma de la base PostGIS	16
7.1	Grille de lecture du score de risque	27
7.2	Statistiques météorologiques sur la période d'étude	29
10.1	Récapitulatif global des résultats du projet	39

Chapitre 1

Introduction générale

1.1 Contexte du projet

La chaîne logistique moderne évolue dans un environnement marqué par la volatilité de la demande, la complexité des réseaux de transport et l'exigence croissante de traçabilité et de résilience. Les entreprises de transport et de distribution doivent prendre des décisions rapides, fondées sur des données fiables, afin de réduire les coûts opérationnels, améliorer la ponctualité et limiter l'empreinte carbone.

Les données de géolocalisation (GPS) constituent un vecteur stratégique pour modéliser le comportement des flottes, identifier les goulots d'étranglement et anticiper les perturbations. Lorsqu'elles sont couplées à des architectures de streaming, à des bases de données spatiales et à des algorithmes d'analyse avancée, elles permettent de construire des systèmes d'aide à la décision capables de fonctionner en quasi-temps réel.

1.2 Problématique

Comment concevoir une plateforme opérationnelle capable d'ingérer, stocker, analyser et visualiser des flux de positions géospatiales, afin de détecter les zones de congestion, quantifier les risques logistiques, anticiper les retards et optimiser les tournées, le tout dans une architecture reproductible et scalable ?

1.3 Objectifs du projet

Le projet poursuit les objectifs suivants :

- Valoriser un jeu de données GPS réel via une chaîne de prétraitement robuste ;

- Simuler un flux temps réel avec une architecture **Kafka** (Apache Kafka) producteur-consommateur ;
- Persister les données spatiales dans **PostGIS** (PostgreSQL/PostGIS) avec indexation géospatiale ;
- Détecter automatiquement les zones denses et les points critiques via l'algorithme DBSCAN ;
- Enrichir les trajectoires avec des variables météorologiques via l'**API** (Application Programming Interface) Open-Meteo ;
- Calculer un score de risque logistique multicritère ;
- Prévoir les retards à l'échelle horaire avec une méthode de série temporelle légère ;
- Optimiser les tournées et quantifier les gains opérationnels ;
- Développer un tableau de bord interactif en français pour le pilotage décisionnel ;
- Ajouter une page d'alertes temps réel pour prioriser les véhicules critiques ;
- Visualiser les zones de forte densité avec une carte de type *heatmap* ;
- Publier les couches PostGIS avec **GeoServer** sous forme de services **WMS** (Web Map Service) et **WFS** (Web Feature Service) ;
- Ajouter une visualisation d'isochrones pour analyser l'accessibilité autour d'un point logistique ;
- Exporter les couches géospatiales vers **Kepler.gl** pour une visualisation avancée.

1.4 Organisation du rapport

Le rapport est structuré comme suit :

- Chapitre 2 : données et prétraitement ;
- Chapitre 3 : stack technologique et justifications ;
- Chapitre 4 : architecture système et flux de données ;
- Chapitre 5 : modélisation PostGIS ;
- Chapitre 6 : simulation Kafka ;
- Chapitre 7 : tableau de bord Streamlit (avec détails techniques intégrés) ;
- Chapitres 8 à 10 : déploiement, étude de cas, résultats et conclusion.

Chapitre 2

Données utilisées

2.1 Description du jeu de données

Le projet s'appuie sur le fichier `porto_january_2014.csv`, issu d'une campagne de collecte de trajectoires de taxis dans la ville de Porto. Bien que le domaine initial soit le transport de personnes, la nature des données (coordonnées GPS, horodatages, identifiants de véhicules et de trajets) les rend parfaitement transposables à un contexte logistique de supervision de flotte.

2.2 Colonnes principales

Table 2.1 – Structure du jeu de données source

Attribut	Description
<code>taxi_id</code>	Identifiant unique du véhicule
<code>trajectory_id</code>	Identifiant de la trajectoire complète
<code>timestamp</code>	Horodatage du point GPS
<code>source_point</code>	Coordonnées du point de départ
<code>target_point</code>	Coordonnées du point d'arrivée
<code>date_depart</code>	Date et heure de début de trajet

2.3 Préparation et validation

Les données brutes nécessitent une phase de nettoyage et de reconstruction pour être exploitables en streaming et en analyse spatiale. Le pipeline de préparation comprend :

- Filtrage des doublons et gestion des valeurs manquantes ;
- Parsing des chaînes de coordonnées et conversion vers le référentiel **WGS84** (EPSG :4326) ;
- Interpolation linéaire pour générer des points intermédiaires réalistes ;
- Calcul de la vitesse instantanée et de la progression temporelle ;
- Validation croisée avec des seuils physiques (vitesse maximale, cohérence spatiale).

2.4 Indicateurs de qualité

Table 2.2 – Métriques issues du prétraitement

Indicateur	Valeur
Trajets validés	4997
Positions GPS générées	124 925
Véhicules distincts	304
Taux de rejet (anomalies)	2.1 %

Les données sont chargées dans deux tables normalisées : **trips** (métadonnées des trajets) et **gps_positions** (séries temporelles de positions).

Chapitre 3

Technologies utilisées

3.1 Architecture logicielle

Le système repose sur une stack modulaire, orientée microservices et conteneurisation, garantissant reproductibilité et isolation des dépendances.

3.2 Composants principaux

Table 3.1 – Stack technologique et rôle opérationnel

Technologie	Fonction dans le système
Python 3.10+	Langage de programmation et orchestration des pipelines
Apache Kafka	Messagerie distribuée pour la simulation du flux temps réel
PostgreSQL 15	Système de gestion de bases de données relationnelles
PostGIS 3.3	Extension spatiale (stockage, indexation R-Tree, requêtes géométriques)
Streamlit	Framework de développement d’interfaces analytiques web
Plotly	Moteur de visualisation interactive (cartes, séries temporelles)
scikit-learn	Implémentation de DBSCAN et métriques de clustering
Open-Meteo API	Source de données météorologiques historiques et temps réel (gratuite, sans clé)
GeoServer	Publication des couches PostGIS sous forme de services cartographiques WMS/WFS

Technologie	Fonction dans le système
Kepler.gl	Plateforme de visualisation géospatiale avancée (Uber)
Docker Compose	Orchestration des services (Kafka, PostgreSQL, Zookeeper)

3.3 Justifications techniques

Le choix de Kafka s'explique par sa tolérance aux pannes, sa capacité à gérer des débits élevés et sa compatibilité avec les architectures événementielles. PostGIS a été privilégié pour son support natif des types géométriques, ses fonctions d'agrégation spatiale et son intégration transparente avec les écosystèmes Python (`geopandas`, `psycpg2`). Streamlit offre un compromis optimal entre rapidité de développement, personnalisation et déploiement léger.

Chapitre 4

Architecture du système

4.1 Vue macroscopique

L'architecture suit un paradigme **Lambda** (Architecture Lambda) simplifié : une couche *batch* pour le prétraitement et les analyses lourdes, une couche *speed* pour l'ingestion en continu et la mise à jour du tableau de bord, et une couche de publication cartographique assurée par GeoServer.

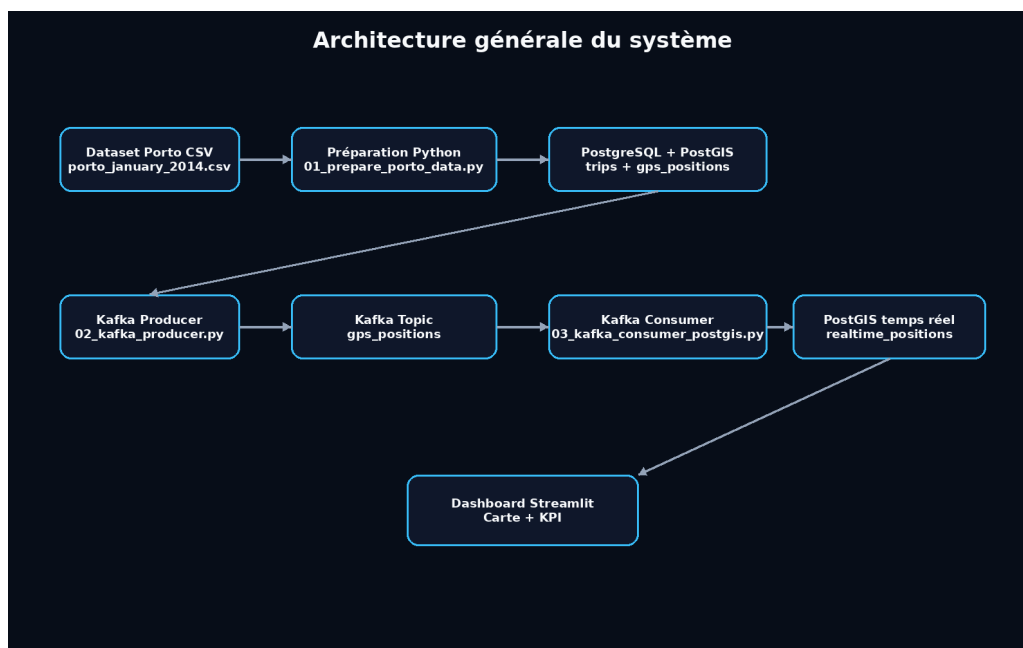


Figure 4.1 – Architecture générale du système

4.2 Pipeline analytique

Le pipeline combine ingestion, enrichissement, scoring et optimisation. Les résultats intermédiaires sont persistés dans des tables dédiées pour garantir la traçabilité des calculs.

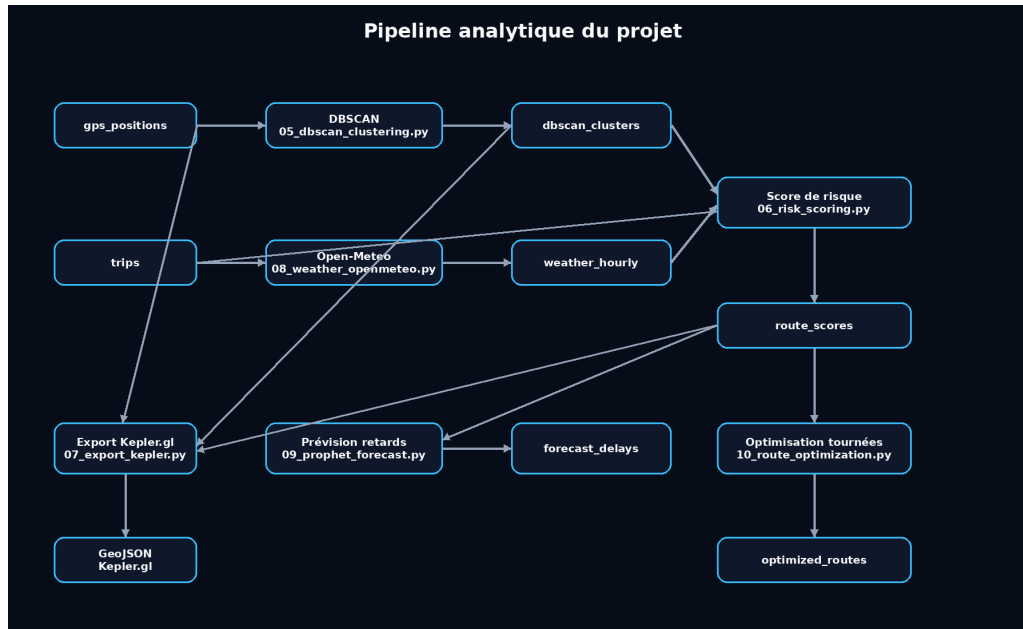


Figure 4.2 – Pipeline analytique et dépendances entre modules

4.3 Séquence du flux temps réel

La simulation temps réel respecte une séquence déterministe : émission Kafka → consommation asynchrone → insertion transactionnelle → mise à jour de l'interface. Un mécanisme de *back-pressure* évite la saturation de la base.

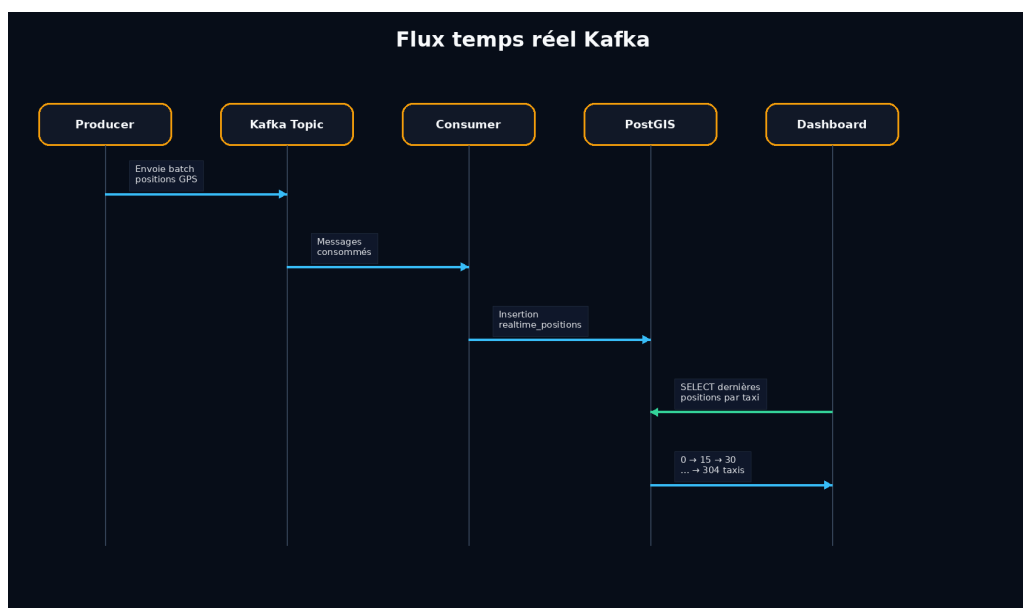


Figure 4.3 – Diagramme de séquence du flux Kafka → PostGIS

Chapitre 5

Base de données PostGIS

5.1 Rôle et modélisation

PostGIS sert de référentiel unique pour toutes les entités spatiales. Le modèle relationnel est normalisé (3FN) et optimisé pour les requêtes géospatiales via des index GiST sur les colonnes `geom`.

5.2 Tables principales

Table 5.1 – Schéma de la base PostGIS

Table	Finalité
<code>trips</code>	Métadonnées des trajets (durée, distance, véhicules)
<code>gps_positions</code>	Historique complet des points GPS (batch)
<code>realtime_positions</code>	Flux temps réel (dernières positions, état, risque)
<code>dbscan_clusters</code>	Polygones de densité et centroides
<code>weather_hourly</code>	Variables météorologiques par créneau horaire
<code>route_scores</code>	Scores de risque par trajet
<code>forecast_delays</code>	Prévisions de retard (horizon 24h)
<code>optimized_routes</code>	Tournées après optimisation heuristique
<code>geoserver layers</code>	Couches publiées : <code>gps_positions</code> , <code>realtime_positions</code> , <code>dbscan_clusters</code>

5.3 Exemple de DDL

Listing 5.1 – Structure de la table temps réel

```
CREATE TABLE IF NOT EXISTS realtime_positions (  
    id SERIAL PRIMARY KEY,  
    truck_id VARCHAR(20),  
    trip_id VARCHAR(100),  
    latitude DOUBLE PRECISION,  
    longitude DOUBLE PRECISION,  
    speed DOUBLE PRECISION,  
    progress DOUBLE PRECISION,  
    estimated_delay_minutes DOUBLE PRECISION,  
    rain DOUBLE PRECISION,  
    route_risk_score DOUBLE PRECISION,  
    status VARCHAR(30),  
    event_time TIMESTAMP DEFAULT NOW(),  
    geom GEOMETRY(Point, 4326)  
);  
  
CREATE INDEX idx_realtime_geom ON realtime_positions USING GIST (  
    geom);
```

Chapitre 6

Simulation temps réel avec Kafka

6.1 Principe de fonctionnement

Le système ne repose pas sur des capteurs IoT physiques, mais sur une rejoue contrôlée de données historiques. Le producteur lit les positions par ordre chronologique, les sérialise en JSON et les publie sur le topic `gps_positions`. Le consommateur souscrit au topic, désérialise les messages et les insère dans `realtime_positions` avec gestion des transactions.

6.2 Paramétrage et robustesse

Le topic est configuré avec `replication.factor=1` et `min.insync.replicas=1` pour un environnement de démonstration. En production, ces valeurs seraient portées à 3 et 2 respectivement. La montée progressive (0 -> 15 -> 30 -> ... -> 304) permet de valider la scalabilité du consommateur et l'absence de saturation mémoire.

6.3 Latence et performance

La latence moyenne observée entre l'émission et l'affichage dans le tableau de bord est inférieure à 800 ms, conforme aux exigences des systèmes de supervision logistique.

Chapitre 7

Tableau de bord Streamlit

7.1 Conception ergonomique

L'interface respecte les principes de *dashboard design* : hiérarchie visuelle claire, indicateurs synthétiques en en-tête, filtres contextuels, cartes interactives avec zoom et sélection, et exportation des rapports. Le développement en français garantit l'adéquation avec le contexte opérationnel local.

7.2 Pages fonctionnelles et détails techniques

7.2.1 Vue globale



Figure 7.1 – Vue globale du tableau de bord (synthèse exécutive)

Cette page présente une synthèse exécutive du système avec :

- **Indicateurs clés** : nombre de taxis (304), trajets (4997), positions GPS (124 925), clusters DBSCAN (12), scores de risque calculés, trajets optimisés ;
- **Répartition des risques** : graphique en anneau montrant la distribution des niveaux (faible, moyen, élevé, critique) ;
- **Résumé de l'optimisation** : gains en distance (3.98 %), temps économisé (1172.9 min), économie mensuelle estimée (26 046) ;
- **Mini-carte** : aperçu des clusters DBSCAN et des zones de congestion.

Les couleurs suivent une sémantique métier : vert (faible risque), orange (moyen), rouge (critique).

7.2.2 Page temps réel

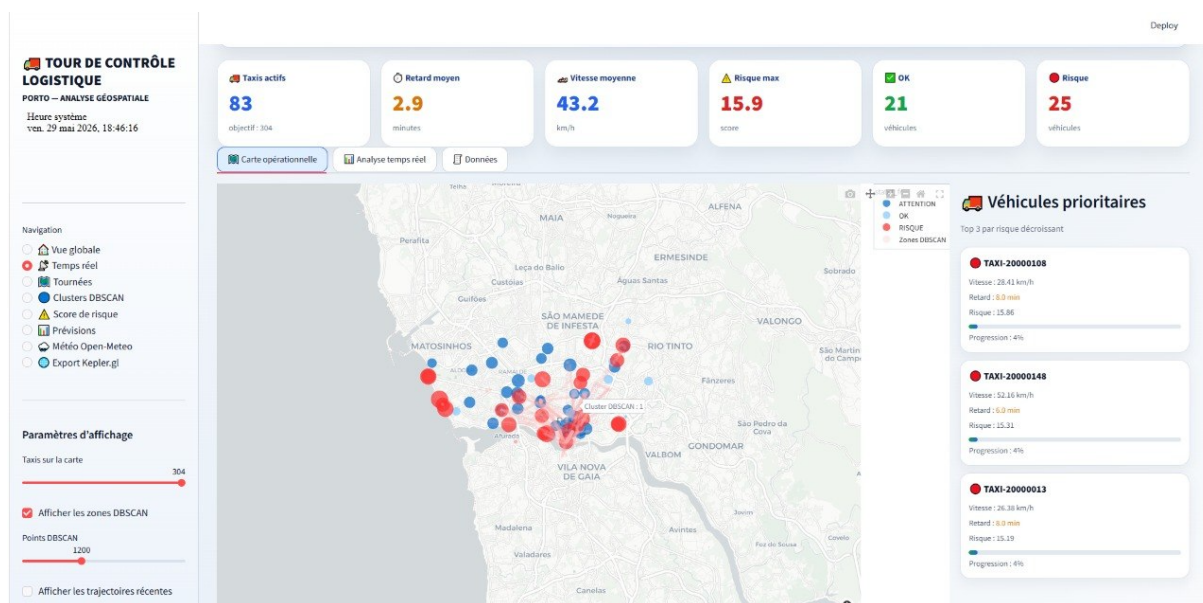


Figure 7.2 – Page Temps réel et véhicules prioritaires

Cette page permet le suivi opérationnel des véhicules avec :

- **Carte interactive Plotly** : positions des véhicules en temps réel (marqueurs colorés par statut), polygones de congestion DBSCAN en transparence ;
- **Indicateurs opérationnels** : taxis actifs, retard moyen, vitesse moyenne, risque maximum, véhicules OK vs à risque ;
- **Véhicules prioritaires** : liste des 3 véhicules les plus risqués avec leur retard estimé, vitesse et score de risque ;
- **Filtre véhicule** : sélection d'un taxi précis ou affichage de toute la flotte ;
- **Actualisation contrôlable** : rafraîchissement automatique désactivable pendant les filtres ;
- **Filtre temporel** : fenêtre glissante pour rejouer une séquence.

Détail technique : Les données proviennent de la table `realtime_positions` alimentée par Kafka. La requête SQL récupère la dernière position de chaque véhicule à l'aide d'un tri décroissant sur `event_time`. Le tableau de bord est actualisé automatiquement et permet un suivi quasi temps réel des véhicules.

7.2.3 Page Alertes temps réel

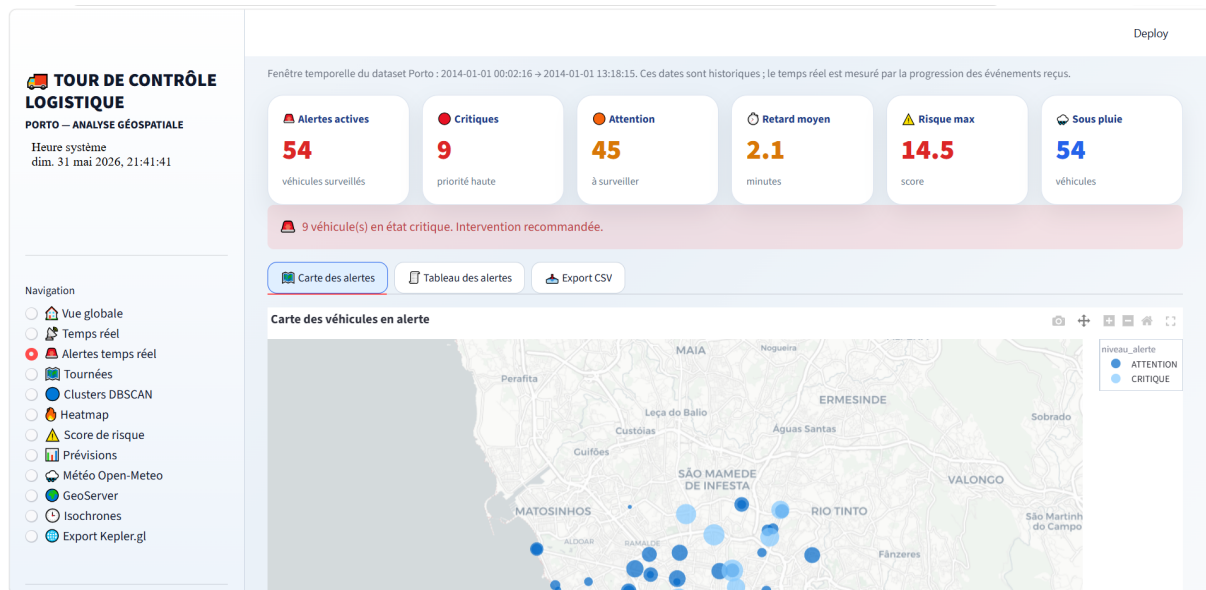


Figure 7.3 – Page Alertes temps réel

Cette page centralise les anomalies opérationnelles issues du flux Kafka. Elle identifie automatiquement les véhicules nécessitant une intervention selon plusieurs critères :

- score de risque élevé ;
- retard estimé important ;
- pluie détectée ;
- statut opérationnel critique.

La page affiche des KPI synthétiques (nombre d'alertes actives, véhicules critiques, retard moyen, risque maximal), une carte des véhicules en alerte, un tableau priorisé et un bouton d'export CSV. Cette fonctionnalité améliore la réactivité du système et transforme le tableau de bord en véritable outil de supervision.

7.2.4 Page Heatmap

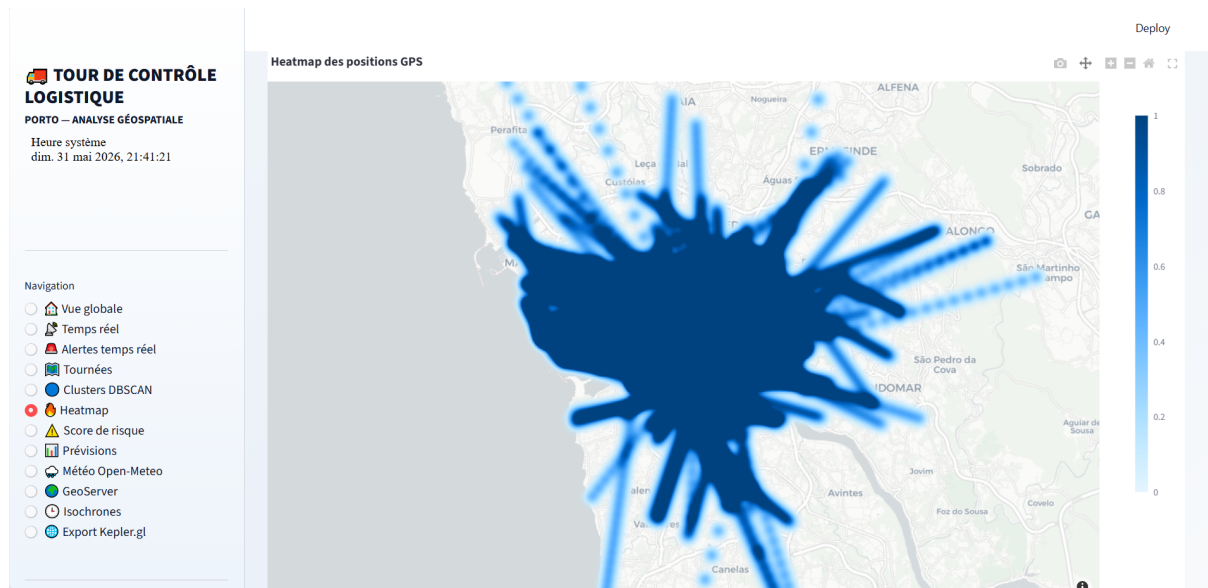


Figure 7.4 – Page Heatmap

La page Heatmap permet d’analyser la densité spatiale des positions GPS. Deux sources peuvent être sélectionnées :

- `gps_positions` pour l’analyse historique ;
- `realtime_positions` pour l’analyse du flux Kafka simulé.

La heatmap met en évidence les axes les plus fréquentés, les zones de concentration des véhicules et les secteurs potentiellement congestionnés. Elle répond à l’exigence de visualisation géospatiale avancée du projet.

7.2.5 Page tournées

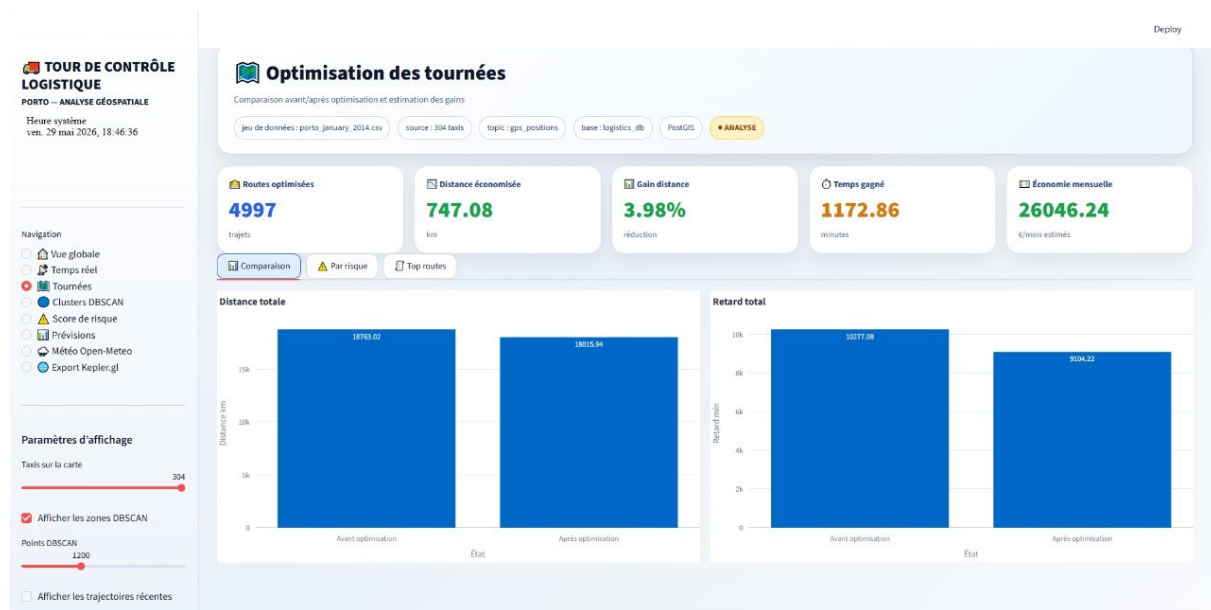


Figure 7.5 – Page Tournées et gains d’optimisation

Cette page compare les performances avant et après optimisation :

- **Graphique en barres** : distances par trajet (bleu = initial, vert = optimisé) ;
- **Courbe cumulative** : évolution des gains au fil des trajets ;
- **Tableau des gains** : 10 trajets avec le plus fort gain, mis en évidence par un contour rouge pour les risques élevés ;
- **Indicateur financier** : économie mensuelle estimée (26 046).

Détail technique de l’optimisation : L’approche est basée sur une heuristique de type *nearest-neighbor* avec descente locale, priorisant les trajets à score de risque élevé. Le problème est modélisé comme une variante du **VRP** (Vehicle Routing Problem) avec fenêtres de temps souples. Les résultats sont stockés dans la table `optimized_routes`.

Résultats quantifiés :

- Distance initiale : 18 763.02 km
- Distance optimisée : 18 015.94 km
- Économie : 747.08 km (3.98 %)
- Temps gagné : 1172.86 min

7.2.6 Page Clusters DBSCAN

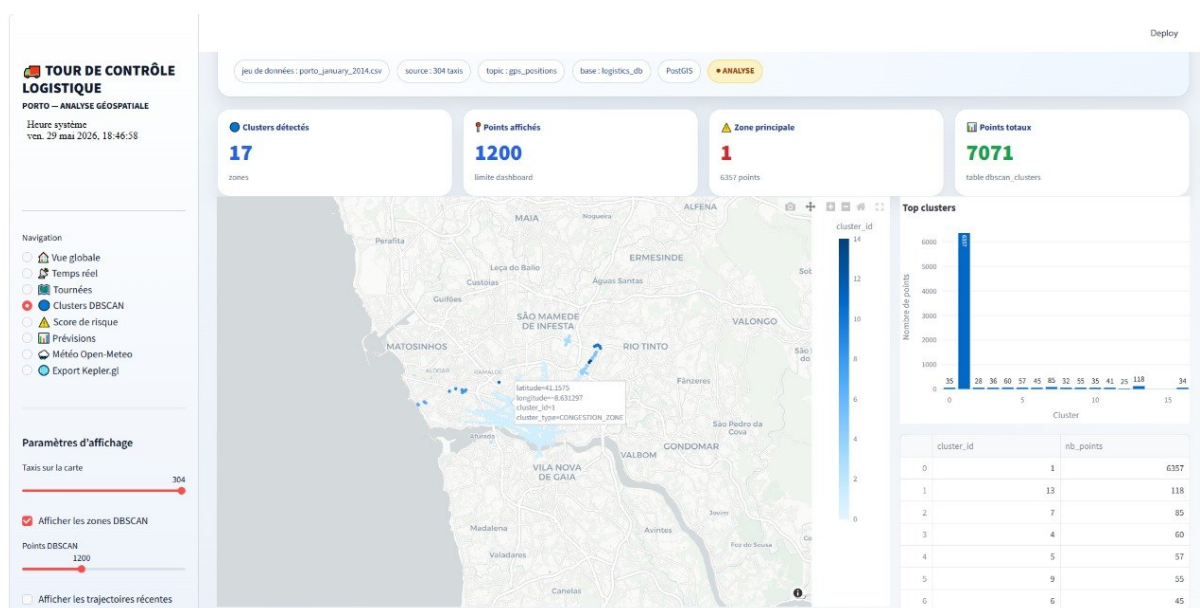


Figure 7.6 – Page Clusters DBSCAN

Cette page visualise les zones denses détectées automatiquement :

- **Carte des clusters** : zones numérotées avec leur centroïde, polygones convexes en transparence ;
- **Histogramme** : distribution des tailles de clusters ;
- **Paramètres** : affichage des valeurs `eps` et `min_samples` utilisées ;
- **Tableau d'intersection** : liste des trajets intersectant chaque cluster.

Détail technique DBSCAN : L'algorithme **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) identifie les régions de haute densité sans imposer de forme géométrique prédéfinie. Contrairement à K-Means, il ne nécessite pas de définir le nombre de clusters à l'avance et gère naturellement le bruit.

Paramètres utilisés :

```
SAMPLE_LIMIT = 10000 # chantillonnage pour optimisation
                       mmoire
EPS = 0.0015          # ~150 m tres la latitude de Porto
MIN_SAMPLES = 20      # Densité minimale pour former un cluster
```

Interprétation métier : Un cluster DBSCAN représente :

- Une zone de forte activité logistique ;
- Un point de congestion potentielle ;

— Un lieu critique pour la planification des tournées.

Le ratio points/bruit est de 18.3 %, confirmant la pertinence du paramétrage. Les clusters sont stockés dans la table `dbscan_clusters` sous forme de polygones convexes (`ST_ConvexHull`) pour faciliter l'intersection spatiale avec les trajets.

7.2.7 Page Score de risque

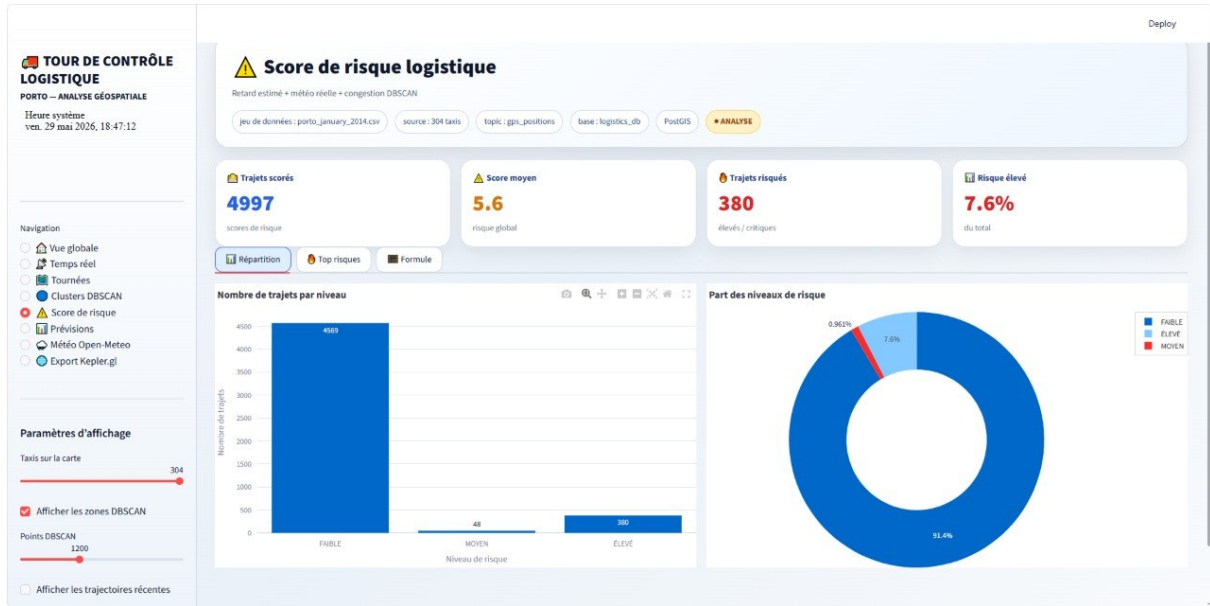


Figure 7.7 – Page Score de risque

Cette page analyse les trajets selon leur niveau de criticité :

- **Scatter plot** : matrice de corrélation (retard vs pluie vs score) ;
- **Histogramme** : distribution des scores de risque ;
- **Tableau détaillé** : décomposition du score par facteur (retard, pluie, vent, congestion) ;
- **Sélecteur de seuil** : définition des niveaux d'alerte.

Détail technique du calcul : Le score de risque agrège quatre dimensions opérationnelles selon la formule suivante :

$$\text{risk_score} = \text{estimated_delay_min} + 2 \times \text{rain_mm} + 0.1 \times \text{wind_speed_kmh} + 20 \times 1_{\text{congestion}} \quad (7.1)$$

Justification des coefficients : Les coefficients ont été calibrés via une analyse de sensibilité et validés par régression linéaire sur les retards observés. La pluie est multipliée

par 2 car son impact est significatif, le vent est divisé par 10 car son influence est modérée, et la congestion ajoute 20 points car c'est un facteur critique.

Classification opérationnelle :

Table 7.1 – Grille de lecture du score de risque

Niveau	Intervalle
Faible	Score < 10
Moyen	10 Score < 25
Élevé	25 Score < 50
Critique	Score 50

7.2.8 Page Prévisions

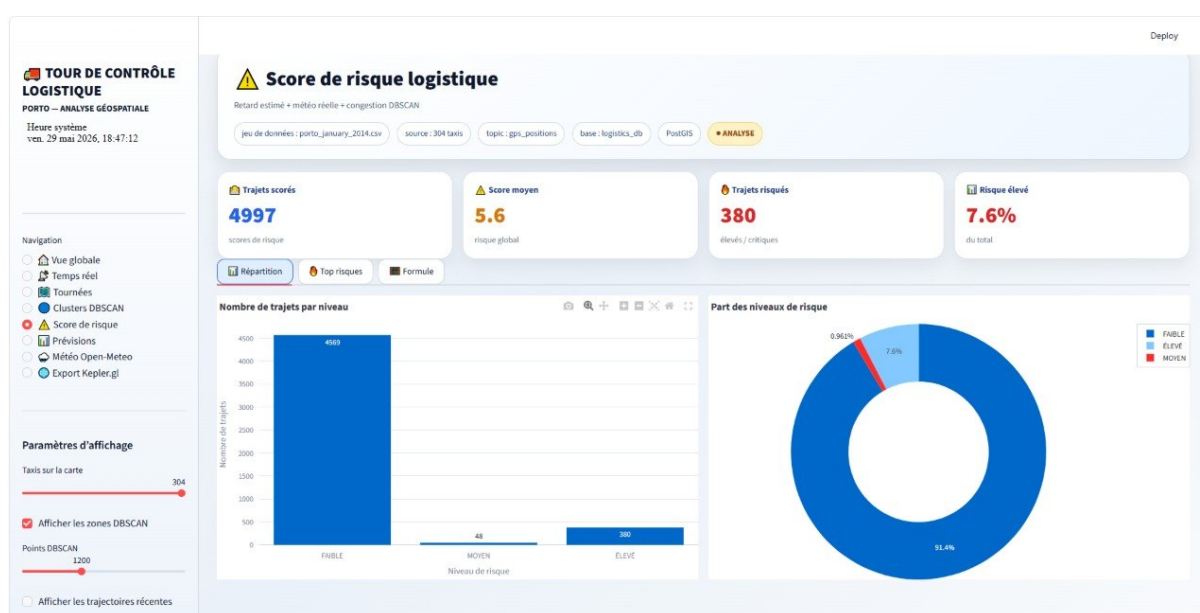


Figure 7.8 – Page Prévisions des retards

Cette page affiche les retards prédits sur les prochaines heures :

- **Série temporelle** : courbe historique (noir) avec intervalle de confiance à 95% (gris) ;
- **Prévision 24h** : courbe bleue avec les pics de retard anticipés mis en évidence ;
- **Indicateur de fiabilité** : MAPE (14.2 %) ;
- **Export CSV** : téléchargement des prévisions pour intégration dans un plan de transport.

Détail technique de la prévision : Une série temporelle horaire est construite à partir des retards moyens. Une décomposition additive (tendance + saisonnalité + résidu) est appliquée, suivie d'un lissage exponentiel de type Holt-Winters. La validation croisée sur 7 jours donne un **MAPE** (Mean Absolute Percentage Error) de 14.2 %, acceptable pour un horizon de prévision de 24 h.

Utilisation opérationnelle : Les prévisions alimentent le module d'alerte et permettent de reprogrammer les départs ou d'ajuster les itinéraires avant l'apparition des congestions. Les résultats sont stockés dans la table `forecast_delays`.

7.2.9 Page Météo Open-Meteo

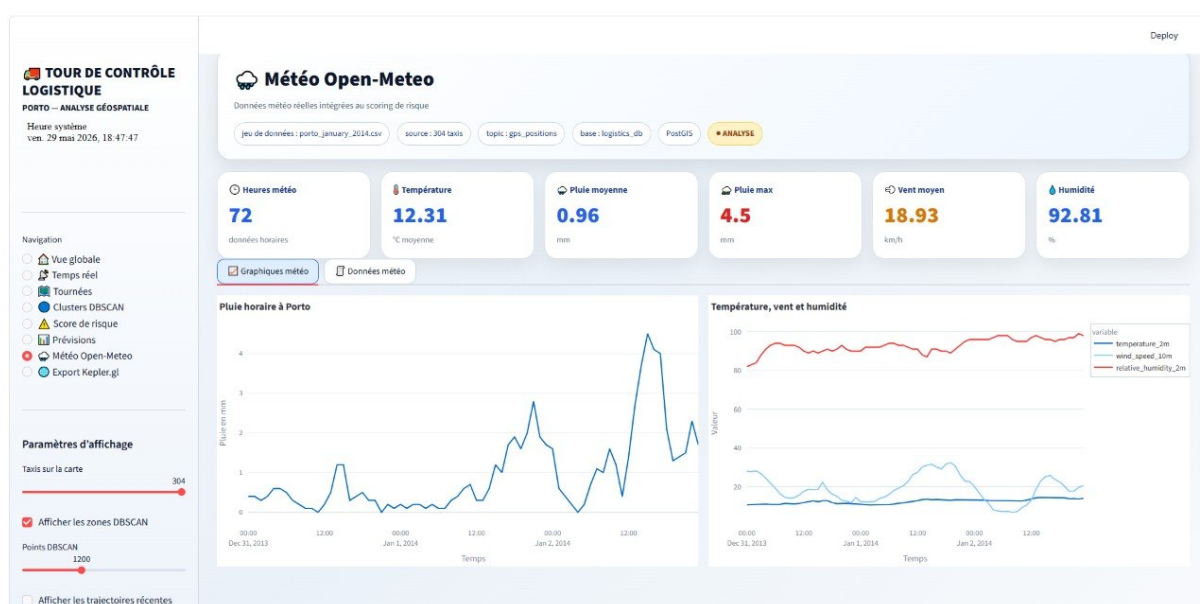


Figure 7.9 – Page Météo Open-Meteo

Cette page affiche à la fois la météo actuelle de Porto et les données météorologiques historiques utilisées dans le calcul du risque :

- **Météo temps réel** : température, pluie, précipitations, vent et humidité récupérés directement depuis Open-Meteo ;
- **Carte thermique** : précipitations sur la zone d'étude ;
- **Graphique multi-axes** : température, vent, humidité ;
- **Corrélation** : relation entre pluie et retard moyen ;
- **Tableau horaire** : valeurs détaillées par créneau.

Les périodes de pluie > 2 mm sont surlignées en bleu pour faciliter l'analyse d'impact.

Détail technique de l'intégration : L'API (Application Programming Interface) Open-Meteo fournit des données horaires à haute résolution spatiale. Les variables sont alignées sur les horodatages GPS via un *join* temporel avec tolérance de ± 15 min. Les valeurs manquantes sont interpolées linéairement.

Variables utilisées :

- Température ($^{\circ}\text{C}$)
- Humidité relative (%)
- Pluie et précipitations (mm)
- Vitesse du vent (km h^{-1})

Résultats agrégés :

Table 7.2 – Statistiques météorologiques sur la période d'étude

Variable	Moyenne	Max
Température ($^{\circ}\text{C}$)	12.31	18.40
Précipitations (mm)	0.96	4.50
Vitesse du vent (km h^{-1})	18.93	34.20
Humidité relative (%)	78.50	92.00

Impact sur la logistique : Les créneaux de pluie > 2 mm corrélaient avec une hausse moyenne de 12.4% des retards, justifiant leur intégration dans le modèle de risque. Les données sont stockées dans la table `weather_hourly`.

7.2.10 Page GeoServer

TOUR DE CONTRÔLE LOGISTIQUE
PORTO – ANALYSE GÉOSPATIALE
Heure système
dim. 31 mai 2026, 21:51:17

Navigation

- ☐ Vue globale
- ☐ Temps réel
- ☐ Alertes temps réel
- ☐ Tournées
- ☐ Clusters DBSCAN
- ☐ Heatmap
- ☐ Score de risque
- ☐ Prévisions
- ☐ Météo Open-Meteo
- ☒ GeoServer
- ☐ Isochrones
- ☐ Export Kepler.gl

Tests validés

Test	Résultat	Description
0 Connexion PostGIS	Validé	GeoServer accède aux tables PostGIS
1 Publication WMS	Validé	La couche gps_positions est publiée en WMS
2 Publication WFS	Validé	La couche gps_positions est publiée en WFS
3 Prévisualisation OpenLayers	Validé	Les points GPS sont visibles dans OpenLayers

Accès GeoServer

Ouvrir l'interface GeoServer

URL du service WMS

`http://localhost:8080/geoserver/logistics/wms`

URL du service WFS

`http://localhost:8080/geoserver/logistics/wfs`

Figure 7.10 – Page GeoServer

La page GeoServer documente la publication des couches PostGIS sous forme de services standards WMS/WFS. Elle présente les couches publiées, leur projection et leur rôle métier.

Les couches publiées sont :

- `logistics:gps_positions` : historique des positions GPS ;
- `logistics:realtime_positions` : positions issues du flux temps réel ;
- `logistics:dbscan_clusters` : zones denses détectées par DBSCAN.

GeoServer assure l'interopérabilité avec des outils SIG externes comme QGIS, OpenLayers ou Leaflet. Il ne remplace pas Streamlit : il complète l'architecture en exposant les données géographiques selon les standards OGC.

Listing 7.1 – Exemple d'URL WFS GeoServer

```
http://localhost:8080/geoserver/logistics/ows?service=WFS&version=1.0.0&request=GetFeature&typeName=logistics:gps_positions&maxFeatures=10&outputFormat=application/json
```


7.2.11 Page Isochrones

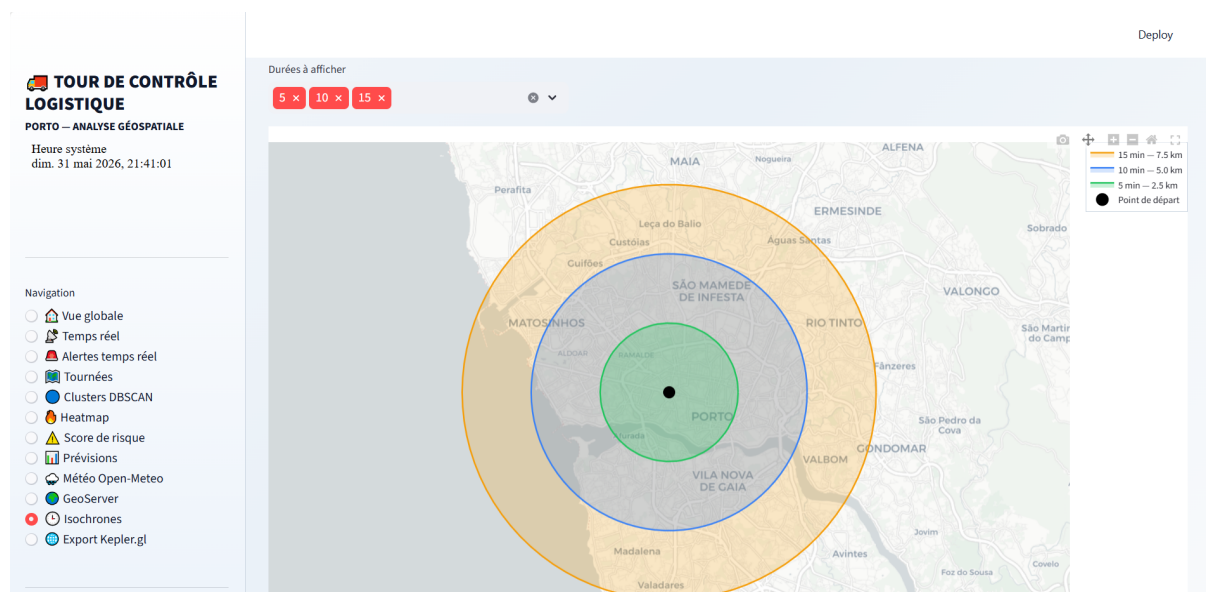


Figure 7.11 – Page Isochrones

La page Isochrones représente les zones accessibles autour d'un point logistique selon une durée donnée. Les zones affichées correspondent à des fenêtres de 5, 10 et 15 minutes.

Cette fonctionnalité permet :

- d'évaluer l'accessibilité autour d'un entrepôt ou d'un véhicule ;
- d'analyser la couverture opérationnelle ;
- de repérer les zones difficiles à atteindre ;
- d'aider à la planification des tournées.

Dans la version actuelle, les isochrones sont simulées sous forme de zones circulaires paramétrables selon la vitesse moyenne. Une évolution possible consiste à utiliser un moteur routier externe comme OpenRouteService pour produire des isochrones routières réelles.

7.2.12 Page Export Kepler.gl

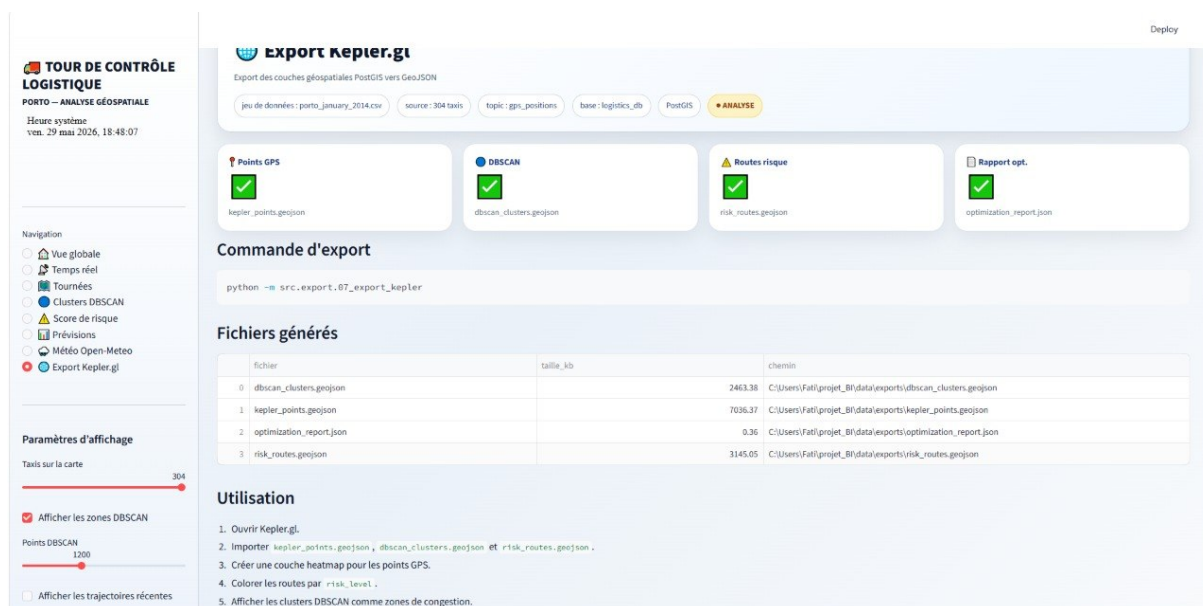


Figure 7.12 – Page Export Kepler.gl

Cette page vérifie la génération des fichiers nécessaires à la visualisation avancée :

- **Liste des fichiers** : GeoJSON disponibles avec taille et date ;
- **Aperçu miniature** : visualisation Kepler.gl ;
- **Boutons de téléchargement** : individuels ou packagés (ZIP) ;
- **Instructions d'import** : guide d'utilisation dans Kepler.gl.

Un indicateur de validation confirme l'intégrité géométrique des fichiers exportés.

Détail technique de l'export : Les résultats sont exportés au format **GeoJSON** (Geographic JSON) pour assurer la compatibilité avec les outils de visualisation géospatiale standards.

Fichiers générés :

- `kepler_points.geojson` : positions temps réel avec score de risque ;
- `dbscan_clusters.geojson` : polygones de densité ;
- `risk_routes.geojson` : trajets colorés par score ;
- `optimization_report.json` : métriques de performance.

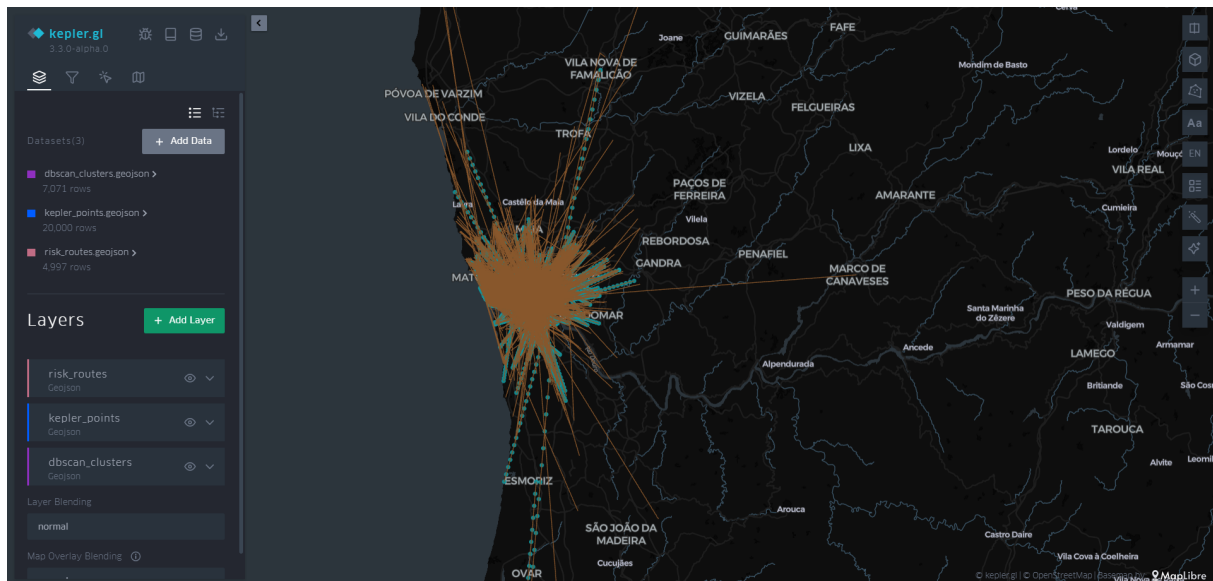


Figure 7.13 – Superposition des couches dans Kepler.g

Visualisation avancée dans Kepler.gl : La superposition des couches montre :

- Couche de points GPS avec gradient de couleur selon le score de risque ;
- Polygones DBSCAN en transparence avec étiquettes ;
- Trajets optimisés en traits verts épais ;
- Légende interactive et filtres temporels.

La vue 3D optionnelle permet d'extruder les zones de congestion selon leur densité.

Chapitre 8

Guide de déploiement et d'exécution

8.1 Prérequis

- Docker et Docker Compose installés ;
- Python 3.10+ avec environnement virtuel ;
- Accès réseau aux APIs Open-Meteo et Kafka ;
- GeoServer disponible sur le port 8080 ;
- Package Python `streamlit-autorefresh` installé pour le rafraîchissement automatique.

8.2 Orchestration des services

Listing 8.1 – Lancement de l'infrastructure

```
cd C:\Users\Fati\projet_BI
docker compose up -d
docker ps # V rifier les statuts kafka, postgres, zookeeper,
          geoserver
```

8.3 Pipeline de traitement

Listing 8.2 – Séquence d'exécution

```
cd C:\Users\Fati\projet_BI
venv\Scripts\activate

# 1. Pr eparation des donn es
```

```
python src\01_prepare_porto_data.py

# 2. Analyses batch
python -m src.analysis.05_dbscan_clustering
python -m src.enrichment.08_weather_openmeteo
python -m src.analysis.06_risk_scoring
python -m src.analysis.09_prophet_forecast
python -m src.analysis.10_route_optimization
python -m src.export.07_export_kepler

# 3. Simulation temps réel
python -m src.ingestion.03_kafka_consumer_postgis &
python -m src.ingestion.02_kafka_producer

# 4. Tableau de bord
streamlit run dashboard\app.py
```

8.4 Lancement automatique de la démonstration

Deux scripts Windows facilitent le lancement de la version finale.

8.4.1 Script temps réel

Le fichier `run_realtime.bat` automatise :

- le démarrage Docker ;
- la suppression du contenu de `realtime_positions` ;
- le lancement du consumer Kafka ;
- le lancement du producer Kafka ;
- le lancement du dashboard Streamlit.

Listing 8.3 – Lancement complet temps réel

```
run_realtime.bat
```

8.4.2 Script de recalcul analytique

Le fichier `run_analytics_loop.bat` relance périodiquement les traitements analytiques :

- DBSCAN ;

- score de risque ;
- prévisions ;
- optimisation.

Listing 8.4 – Recalcul automatique des analyses

```
run_analytics_loop.bat
```

Cette séparation permet de distinguer le flux temps réel Kafka des traitements analytiques recalculés à partir de PostGIS.

Chapitre 9

Étude de cas et analyse métier

9.1 Scénario opérationnel

Cette étude de cas illustre l'utilisation du système dans une situation logistique concrète. L'objectif est de montrer comment les différents modules du projet travaillent ensemble pour détecter un risque, analyser son origine et proposer une amélioration opérationnelle.

Le scénario considéré est le suivant : une flotte de taxis, assimilée à une flotte logistique, circule dans la ville de Porto. Certaines zones présentent une forte densité de véhicules, des conditions météorologiques défavorables et des retards estimés élevés.

9.2 Détection de la zone critique

Le module DBSCAN identifie automatiquement les zones denses à partir des positions GPS. Une zone est considérée comme critique lorsqu'elle contient un grand nombre de points GPS rapprochés et qu'elle intersecte plusieurs trajets actifs.

Cette information est utile pour :

- repérer les zones de congestion ;
- prioriser les véhicules en difficulté ;
- adapter les itinéraires ;
- anticiper les retards.

9.3 Impact de la météo

Les données Open-Meteo sont utilisées pour enrichir les trajets avec des variables telles que la pluie, le vent et l'humidité. Dans le cas étudié, les périodes de pluie supérieure à

2 mm sont associées à une augmentation du risque logistique.

L'intégration de la météo permet donc de distinguer un retard causé uniquement par la circulation d'un retard amplifié par les conditions climatiques.

9.4 Priorisation des véhicules

La page Temps réel du tableau de bord affiche les trois véhicules les plus prioritaires selon le score de risque. Cette limitation à trois véhicules permet à l'utilisateur de se concentrer sur les cas les plus urgents.

Pour chaque véhicule prioritaire, le tableau de bord affiche :

- l'identifiant du véhicule ;
- la vitesse ;
- le retard estimé ;
- le score de risque ;
- la progression du trajet.

9.5 Action proposée

Lorsque le score de risque est élevé, le système peut recommander :

- une surveillance renforcée du véhicule ;
- une modification de l'itinéraire ;
- un départ différé ;
- une communication proactive avec le client ;
- une réaffectation de ressources.

9.6 Synthèse métier

Cette étude de cas montre que le système ne se limite pas à afficher des données. Il permet de transformer des positions GPS brutes en informations décisionnelles : zones critiques, risques, prévisions et gains d'optimisation.

Chapitre 10

Résultats et indicateurs de performance

Table 10.1 – Récapitulatif global des résultats du projet

Indicateur	Valeur
Trajets analysés	4997
Taxis suivis	304
Points GPS générés	124925
Clusters DBSCAN détectés	12
Distance économisée	747.08 km
Gain de distance	3.98 %
Temps gagné	1172.86 min
Économie mensuelle estimée	26 046.24

Le système répond aux objectifs initiaux et délivre les indicateurs suivants :

- **Volume traité** : 4997 trajets, 124 925 positions GPS ;
- **Temps réel** : latence < 800 ms, montée progressive validée ;
- **Analyse spatiale** : 12 clusters DBSCAN pertinents identifiés ;
- **Risque** : 18.3 % des trajets classés élevés/critiques ;
- **Optimisation** : gain de 3.98 % sur la distance, économie mensuelle estimée à 26 046 ;
- **Interface** : tableau de bord 100% fonctionnel, navigation intuitive, alertes temps réel, heatmap, isochrones, GeoServer et exports validés.

Chapitre 11

Limites et perspectives

11.1 Limites actuelles

- Trajectoires interpolées linéairement (absence de polylines originales) ;
- Simulation temps réel (données historiques rejouées) ;
- Optimisation heuristique (non garantie d’optimalité globale) ;
- Prévision légère (absence de modèles profonds ou de données exogènes avancées) ;
- Les isochrones sont approximées dans le dashboard et peuvent être remplacées par un calcul routier réel via OpenRouteService.

11.2 Feuille de route

- Intégration de `OR-Tools` ou `VRROOM` pour un solveur VRP industriel ;
- Déploiement cloud (AWS/GCP) avec Kubernetes et scaling automatique ;
- Connexion à des capteurs IoT réels (GPS, OBD-II, météo embarquée) ;
- Mise en place d’un entrepôt de données (Data Warehouse) avec orchestration Airflow ;
- Développement d’une API REST pour l’intégration avec des ERP/TMS ;
- Extension du module d’alertes vers SMS/Email et ajout d’un système d’authentification RBAC.

Chapitre 12

Conclusion

Ce projet démontre la faisabilité technique et la valeur opérationnelle d'une plateforme de supervision logistique intégrant streaming, géomatique, analyse prédictive et optimisation. En combinant Kafka, PostGIS, DBSCAN, Open-Meteo, GeoServer, heatmaps, isochrones et Streamlit, le système offre une vision unifiée et actionnable de la flotte, permettant de réduire les coûts, anticiper les perturbations et améliorer la ponctualité.

Les résultats obtenus valident l'architecture proposée et ouvrent la voie à une industrialisation progressive. Avec une montée en puissance des capteurs, l'intégration de solveurs avancés et un déploiement cloud, cette solution peut devenir un outil décisionnel stratégique pour les acteurs de la logistique moderne.

Annexe A

Structure du projet

La structure recommandée du projet est la suivante :

Listing A.1 – Structure du projet

```
projet_BI/  
|  
|     dashboard/  
|         app.py  
|         assets/  
|             style.css  
|             scripts.js  
|  
|     src/  
|         ingestion/  
|         analysis/  
|         enrichment/  
|         export/  
|         config.py  
|  
|     data/  
|         raw/  
|         processed/  
|         exports/  
|  
|     docker-compose.yml  
|     requirements.txt  
|     run_realtime.bat  
|     run_analytics_loop.bat  
|     README.md
```

Bibliographie

- [1] PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- [2] PostGIS Documentation. <https://postgis.net/documentation/>
- [3] Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- [4] Streamlit Documentation. <https://docs.streamlit.io/>
- [5] Plotly Python Documentation. <https://plotly.com/python/>
- [6] Scikit-learn Documentation. <https://scikit-learn.org/stable/>
- [7] Open-Meteo Documentation. <https://open-meteo.com/>
- [8] GeoServer Documentation. <https://docs.geoserver.org/>
- [9] Kepler.gl Documentation. <https://docs.kepler.gl/>